Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Análise de Algoritmos (Parte I)

Departamento de Informática (DI)
Centro Tecnológico (CT)
Universidade Federal do Espírito Santo (UFES)

(Material baseado nos slides dos Professores Eduardo Zambon e Giovanni Comarela)

Introdução

- A análise de algoritmos é uma ferramenta fundamental para o entendimento adequado dos programas.
- **Aula de hoje**: apresentação dos diferentes métodos que serão usados no curso para analisar/comparar a performance de algoritmos.
- **Objetivos**: entender a ordem de crescimento de funções, análise assintótica e métodos de medição experimental de tempo de execução de programas.

Referências

Capítulo 2 do livro *Algorithms in C – Principles of Algorithm Analysis*
R. Sedgewick

Part I

Introdução

Razões para se analisar algoritmos

Análise de algoritmos neste curso:

- Predição de desempenho.
- Comparação de algoritmos.
- Provimento de garantias no comportamento de programas.

Análise de algoritmos em um curso específico/dedicado:

- Provimento de garantias no comportamento de programas.
- Compreensão do embasamento teórico de complexidade.

Part II

Medições

Exemplo: 3-SUM

3-SUM: Dados *N* inteiros distintos, quantas triplas quando somadas são iguais a zero?

```
$ cat 8ints.txt
30 -40 -20 -10 40 0 10 5

$ ./a.out 8 < 8ints.txt
4
```

a[i]	a[j]	a[k]	sum
30	-40	10	0
30	-20	-10	0
-40	40	0	0
-10	0	10	0

Algoritmo força bruta: verifica cada tripla.

```
int count(int *a, int N) {
  int count = 0;
  for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
      for (int k = j+1; k < N; k++)
        if (a[i] + a[j] + a[k] == 0)
          count++;
  return count;
}
```

Ignora *overflow* de inteiros para simplificar.

Medindo o tempo de execução

Q: Como medir o tempo de execução de um programa?

A: Usando a biblioteca `time.h`.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "3sum.h"

int main(int argc, char *argv[]) {
  int N = atoi(argv[1]);
  int *a = malloc(N * sizeof(int));
  for (int i = 0; i < N; i++) {
    scanf("%d", &a[i]);
  }
  clock_t start, stop;
  start = clock();
  int c = count(a, N);
  stop = clock();
  double time_taken = ((double) stop - start) / CLOCKS_PER_SEC;
  printf("%d\n", c);
  printf("Elapsed time: %.3f\n", time_taken);
  free(a);
}
```

Análise empírica

Executar o programa para entradas de diferentes tamanhos e medir o tempo de execução.

N	1,000	2,000	4,000	8,000	16,000
tempo (s)	0.348	2.676	21.272	170.150	?

Análise de dados



Previsão e validação

É possível determinar uma curva que passa pelos pontos do gráfico. Exemplo: `https://mycurvefit.com/` .

Aplicando regressão cúbica aos pontos do gráfico anterior, temos que o tempo de execução $T(N)$ é dado por:

$$T(N) = 0.332893N^3 - 0.006917N^2 + 0.018500N + 0.003524$$

para N múltiplo de 1K.

Análise empírica

Efeitos independentes do sistema:

- São características do algoritmo e da entrada que não são influenciadas pelo sistema aonde o programa executa.
- Determinam os expoentes da função $T(N)$.

Efeitos dependentes do sistema:

- Determinam as constantes da função $T(N)$.
- Totalmente influenciados pelas características do sistema: hardware, SO, compilador, carga, etc.

Má notícia: é difícil obter medidas precisas.
Boa notícia: em geral é fácil realizar um grande número de experimentos.

Part III

Modelos Matemáticos

Modelos matemáticos para tempo de execução

Tempo total de execução: somatório de custo $\times$ frequência para todas as operações do programa.

- É necessário analisar o programa para se determinar o conjunto de operações.
- Custo depende da máquina, compilador, etc.
- Frequência depende do algoritmo e dados de entrada.

Custo das operações básicas

Observação: A maioria das operações primitivas leva tempo constante.

operação	exemplo	tempo
declaração de variável	`int a`	c_1
comando de atribuição	`a = b`	c_2
comparação de inteiros	`a < b`	c_3
acesso em *array*	`a[i]`	c_4
alocação de *array* 1D	`calloc (int[N])`	$c_5 N$
alocação de *array* 2D	`calloc (int[N][N])`	$c_6 N^2$

- Cuidado: Operações não-primitivas geralmente levam mais do que tempo constante.
- Erro de principiante: abusar da concatenação de *strings*.

Exemplo: 1-SUM

Q: Quantas instruções como uma função da entrada de tamanho *N*?

```
int count = 0;
for (int i = 0; i < N; i++)
  if (a[i] == 0)
    count++;
```

operação	frequência
declaração de variável	
comando de atribuição	
comparação $<$	
comparação $==$	
acesso em *array*	
incremento	

Exemplo: 1-SUM

Q: Quantas instruções como uma função da entrada de tamanho N?

```
int count = 0;
for (int i = 0; i < N; i++)
  if (a[i] == 0)
    count++;
```

operação	frequência
declaração de variável	2
comando de atribuição	2
comparação $<$	$N + 1$
comparação $==$	N
acesso em *array*	N
incremento	entre N e $2N$

Exemplo: 2-SUM

Q: Quantos acessos ao *array*?

```
int count = 0;
for (int i = 0; i < N; i++)
  for (int j = i+1; j < N; j++)
    if (a[i] + a[j] == 0)
      count++;
```

Devemos determinar quantas vezes o *loop* interno é executado. Isso pode ser feito como na matriz abaixo (assumindo $N = 5$).

$i \setminus j$	0	1	2	3	4	
0		X	X	X	X	$(N-1) = 4$
1			X	X	X	$(N-2) = 3$
2				X	X	$(N-3) = 2$
3					X	$(N-4) = 1$
4						$(N-5) = 0$

$0 + 1 + \ldots + (N-1) = {}^1\!/_2 N(N-1) \times 2$ acessos $= N(N-1)$.

Exemplo: 2-SUM

Q: Quantas instruções como uma função da entrada de tamanho N?

```
int count = 0;
for (int i = 0; i < N; i++)
  for (int j = i+1; j < N; j++)
    if (a[i] + a[j] == 0)
      count++;
```

operação	frequência
declaração de variável	$N + 2$
comando de atribuição	$N + 2$
comparação $<$	$1/2(N + 1)(N + 2)$
comparação $==$	$1/2N(N - 1)$
acesso em *array*	$N(N - 1)$
incremento	entre $1/2N(N - 1)$ e $N(N - 1)$

Muito tedioso contar todas as operações de forma exata.

Simplificando os cálculos

"É conveniente termos uma medida da quantidade de trabalho envolvida em um processo computacional, mesmo quando essa medida for bastante grosseira. Podemos querer contar a quantidade de vezes que variadas operações elementares são aplicadas a todo o processo e atribuí-las pesos. [...] No caso de computações com matrizes, a maior parte do trabalho envolve multiplicações e acessos à matriz, portanto vamos tentar contar apenas essas operações."

ROUNDING-OFF ERRORS IN MATRIX PROCESSES

By A. M. TURING

(*National Physical Laboratory, Teddington, Middlesex*)

[Received 4 November 1947]

SUMMARY

A number of methods of solving sets of linear equations and inverting matrices are discussed. The theory of the rounding-off errors involved is investigated for some of the methods. In all cases examined, including the well-known 'Gauss elimination process', it is found that the errors are normally quite moderate: no exponential build-up need occur.



Simplificação 1: modelo de custo

Modelo de custo: Usar alguma operação básica como um representante do tempo de execução.

```
int count = 0;
for (int i = 0; i < N; i++)
  for (int j = i+1; j < N; j++)
    if (a[i] + a[j] == 0)
      count++;
```

Exemplo de modelo de custo: acessos em *array*.

operação	frequência
declaração de variável	$N + 2$
comando de atribuição	$N + 2$
comparação $<$	$1/2(N + 1)(N + 2)$
comparação $==$	$1/2N(N - 1)$
acesso em *array*	$N(N - 1)$
incremento	entre $1/2N(N - 1)$ e $N(N - 1)$

Simplificação 2: notação til ($\sim$)

- Estimar o tempo de execução (ou uso de memória) como uma função do tamanho da entrada N.
- Ignorar os termos de mais baixa ordem.
  - Quando N é grande, esses termos são desprezíveis.
  - Quando N é pequeno, não faz diferença.

Exemplo:

$$1/6 N^3 - 1/2 N^2 + 1/3 N \sim 1/6 N^3$$

Para $N = 1000$: 166.67 milhões vs. 166.17 milhões.

Definição matemática: $f(N) \sim g(N)$ significa

$$\lim_{N \to \infty} \frac{f(N)}{g(N)} = 1 \quad .$$

Exemplo: 2-SUM com notação $\sim$

Q: Aproximadamente quantos acessos ao *array* como uma função do tamanho de entrada N?

```
int count = 0;
for (int i = 0; i < N; i++)
  for (int j = i+1; j < N; j++)
    if (a[i] + a[j] == 0)
      count++;
```

A: $2(0 + 1 + \ldots + (N-1)) = 2\binom{N}{2} = N(N-1) \sim N^2$ acessos.

Exemplo: 3-SUM com notação $\sim$

Q: Aproximadamente quantos acessos ao *array* como uma função do tamanho de entrada N?

```
int count = 0;
for (int i = 0; i < N; i++)
  for (int j = i+1; j < N; j++)
    for (int k = j+1; k < N; k++)
      if (a[i] + a[j] + a[k] == 0)
        count++;
```

A: O *loop* mais interno é executado $\binom{N}{3}$ vezes. O total de acessos ao *array* é

$$3\binom{N}{3} = 3\left(\frac{N(N-1)(N-2)}{3!}\right) \sim {}^1\!/_2 N^3 \text{acessos.}$$

Modelos matemáticos para tempo de execução

Em princípio, modelos matemáticos precisos estão disponíveis.

Na prática:

- As fórmulas podem ser complicadas.
- Matemática avançada pode ser necessária.
- $\Rightarrow$ Modelos exatos ficam melhor na mão de especialistas.

Além disso:

- Custos das operações dependem do hardware e do compilador.
- Frequência das operações dependem do algoritmo e da entrada.

$\Rightarrow$ Vamos usar modelos aproximados neste curso:

$$T(N) \sim cN^3 \quad .$$

Part IV

Classificação das Ordens de Crescimento

Classificação das ordens de crescimento

Definição: Se $f(N) \sim c\,g(N)$ para alguma constante $c > 0$, então a ordem de crescimento de $f(N)$ é $g(N)$.

- Ignora os termos de mais baixa ordem (como na notação $\sim$).
- Também ignora o coeficiente do maior termo.

Exemplo: a ordem de crescimento do tempo de execução do código abaixo é N^3.

```
int count = 0;
for (int i = 0; i < N; i++)
  for (int j = i+1; j < N; j++)
    for (int k = j+1; k < N; k++)
      if (a[i] + a[j] + a[k] == 0)
        count++;
```

Simplificações:

- O programa é da ordem de N^3.
- O programa é cúbico.

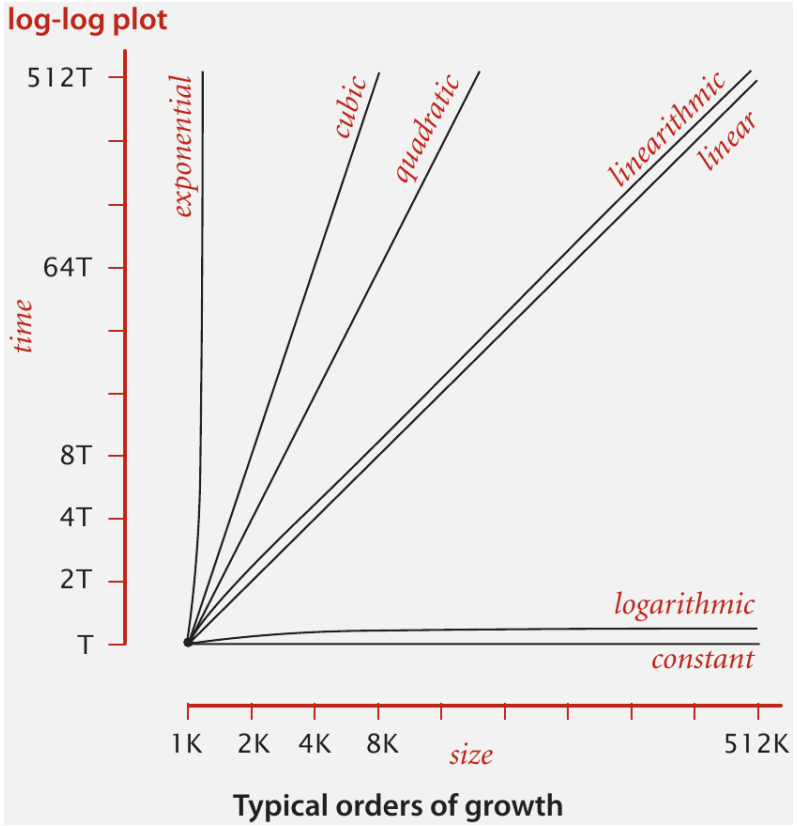
Uso típico: Para indicar tempos de execução aonde o maior coeficiente depende da máquina, compilador, etc.

Classificação das ordens de crescimento

Boas notícias: O conjunto de funções

$$1,\ \log N,\ N,\ N \log N,\ N^2,\ N^3 \ \text{ e } \ 2^N$$

é suficiente para descrever a ordem de crescimento da maioria dos algoritmos mais comuns.



Busca binária

Objetivo: Dados um *array* ordenado e uma chave, determinar o índice da chave no *array*.

Busca binária: Compare a chave com a entrada no meio do vetor.

- Se for menor, vá para esquerda.
- Se for maior, vá para direita.
- Se for igual, chave encontrada.

Ver arquivo `14DemoBinarySearch.pdf`.
(`https://algs4.cs.princeton.edu/lectures/demo/14DemoBinarySearch.pdf`)

Busca binária - Implementação

```
int bin_search(int *a, int sz, int key) {
  int lo = 0, hi = sz-1;
  while (lo <= hi) {
    int mid = lo + ((hi - lo) / 2);
    if      (key < a[mid]) hi = mid - 1;
    else if (key > a[mid]) lo = mid + 1;
    else return mid;
  }
  return -1;
}
```

Um algoritmo $N^2 \lg N$ para 3-SUM

Proposição: Busca binária usa no máximo $1 + \lg N$ comparações para buscar uma chave em um *array* ordenado de tamanho N. Prova: Em breve.

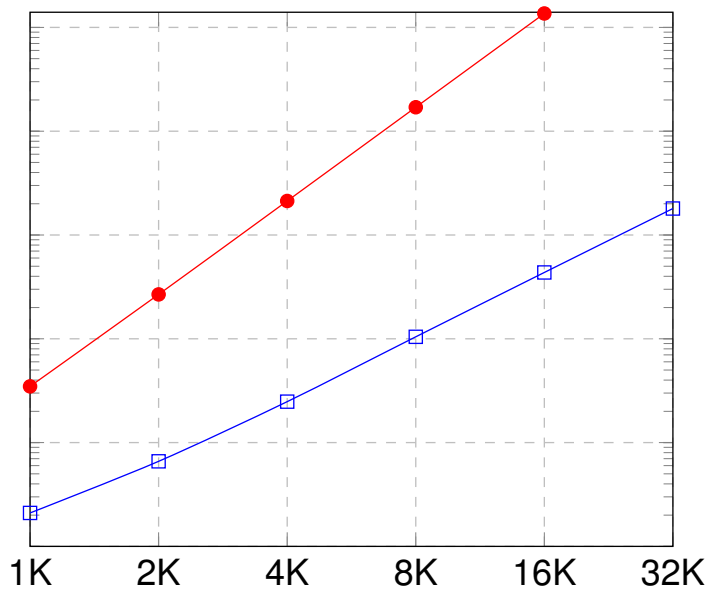
```
int count(int *a, int N) {
  ins_sort(a, N);
  int count = 0;
  for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++) {
      int ak = -(a[i] + a[j]);
      if (ak <= a[j]) continue; // Evitar repeticao
      if (bin_search(a, N, ak) != -1)
        count++;
    }                          // In: -40 -20 -10 0 5 10 30 40
  return count;                // Testado: -40 10  30
}                              // Pulado:   10 30 -10
```

Análise: Ordem de crescimento é $N^2 \lg N$.

Comparando programas

Hipótese: O algoritmo baseado em ordenação para 3-SUM (T_2) é significativamente mais rápido na prática do que o algoritmo força-bruta (T_1).

N	1,000	2,000	4,000	8,000	16,000	32,000
T_1	0.348	2.676	21.272	170.15	1,362.06	?
T_2	0.021	0.066	0.248	1.045	4.356	17.989



Em geral: melhor ordem de crescimento $\Rightarrow$ mais rápido.

Tipos de análises

Melhor caso: limite inferior do custo.

- Determinado pela entrada "mais fácil".
- Provê uma meta para todas as entradas.

Pior caso: limite superior do custo.

- Determinado pela entrada "mais difícil".
- Provê uma garantia para todas as entradas.

Caso médio: custo esperado para uma entrada aleatória.

- Requer um modelo para a entrada "aleatória".
- Provê uma forma de prever o desempenho.

Ex. 1: Acessos ao *array* para 3-SUM força bruta.

Melhor	$\sim \frac{1}{2} N^3$
Médio	$\sim \frac{1}{2} N^3$
Pior	$\sim \frac{1}{2} N^3$

Ex. 2: Comparações na busca binária.

Melhor	~ 1
Médio	$\sim \lg N$
Pior	$\sim \lg N$